OpenGL Intro:
Basic Rendering and Transformations
The Sun, Earth and Moon

CSC4029Z Assignment 1

April, 2026

1 Introduction

This is your first practical in OpenGL which is designed to get you familiar with OpenGL before you delve into shaders. The aim is to teach you some fundamentals, like setting up a window and default camera, and populating the scene with objects. After you have the objects set up, you need to provide keyboard controls to start/stop the update of the scene parameters (see later) that control how your OpenGL scene is rendered.

You should implement your solution based on either the C/C++ or Python code skeletons provided as part of the assignment (or use only the libraries/modules used in the frameworks and be able to explain all changes made).

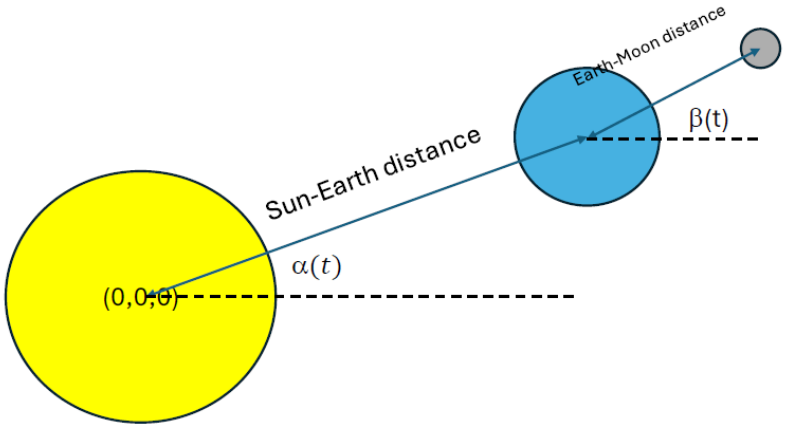
For the C++ OpenGL assignment option you will make use of the SDL library to create a window and respond to input. SDL is a library that provides cross-platform access to functionality commonly required by real-time interactive applications (such as window creation, input handling, file I/O etc). SDL is popular for games development, having been used in FTL, World of Goo, and Portal, amongst others.

Detailed instructions on how to setup SDL on your machine are available on their wiki (wiki.libsdl.org), however broadly speaking, you will need to install graphics drivers, and the libraries/headers for **glew** and **SDL**. For this practical you will be using **OpenGL version 3.2 or higher**.

The Python assignment option requires setting up a Python virtual environment and installing the appropriate modules/packages. Instructions are included in the Python code skeleton zip file. The included makefile will also install useful packages such as *pyrr* into the virtual environment and, should you wish to install any other packages (such as *argparse* or *matplotlib*), you can just include them in the *requirements.txt* file and rebuild your virtual environment. **Do not touch the post_requirements.txt file**, it is needed to correctly install *PyOpenGL-accelerate*. Note that I cannot offer much guidance on using Python since I have always used C/C++ for OpenGL.

2 Requirements

For this assignment you will set up and render a simple representation of the Sun-Earth-Moon system, See Figure below:



Sizes (radii): Sun >> Earth >> Moon

Each of these is a sphere model, scaled and with a single representative colour (I suggest yellow, blue/aqua and grey). The planets need to be scaled such that the Earth is much smaller than the Sun, and the Moon is much smaller than the Earth. The scale should be a little exaggerated so you can see all the spheres clearly on your screen at the same time. You will also need to set distances from the centres of the Sun-Earth and from the Earth-Moon. You can choose these for visual impact. By default, the Sun is assumed to lie on the World Space origin (0,0,0), with the other planets offset from this position. You will need to change this since we are assuming the camera is located at the origin too.

2.1 Basic Window and Camera Setup

Your first task is to set up the scene by loading and displaying an OBJ format sphere model (found under Resources). Start by just displaying the "Sun" model – you can then add the other models once you have figured this part out. Setup code has been provided for you to manage object loading and window setup. You will need to edit this code to add in the required functionality.

In this assignment, we are not changing the camera: the default OpenGL camera COP is located at the origin, (0,0,0), looking down the negative Z axis, so just place your objects accordingly - down the -Z axis - so the camera can image your scene. Make sure the objects are placed far enough away that the camera (and its front clipping plane) does not end up inside the object. Of course, to be imaged, your scene objects must be inside the camera view frustum (the front clipping plane is the projection plane usually and the far clipping plane must be far enough away that no part of the models is clipped off. Adjust the cameras parameters appropriate for field of view etc You can use either **glm** for C++/C, or Python's **pyrr** to generate the camera matrix for you.

The framework code provided should (when appropriate compiled/configured) display a

window within which a single white triangle is drawn. If you are using the Python skeleton code, you can use pyrr to setup your camera and perform matrix transformation (See Section 6). Using pure Python (i.e. lists rather than numpy arrays) to do all of this is going to be unnecessarily complex and slow.

2.2 Loading an Object

The code framework includes code to load OBJ files (as well as sample OBJ files to load). Use this code to load a model at startup, and have it draw to the screen. The provided code includes a number of useful methods to retrieve model vertex positions, texture coordinates and normals (only vertex positions are needed for Assignment 1). NOTE: several OBJ models are provided, but some of these may not have normals per vertex/face or may be incorrectly "wound" (see lectures). In this case, we want our planets to be **spheres**, so just use the provided sphere model (which should be correct). There was a bug in the OBJ loader, but I think that is now fixed (fingers crossed).

2.3 Model Colour

You will not do any real shading in this assignment. Each model simply needs (one) unique colour. There is a basic shader in the example code that sets a single color per fragment (white), so just use that to set your own colour for each sphere. Proper shading and texturing will be implemented in Assignment 2.

2.4 Animation

As you may note from the Figure, the sun remains stationary and the Earth rotates about the sun, with an angle that depends on some global system time, t. If we assume a default camera (so View Matrix = Identity), then the Earth should rotate about the Sun in the x-y plane (remember standard camera viewpoint is at origin looking down -Z). In effect, you do not keep track of t explicitly, you simply have some initial angle α (likely 0), and then add a small angle delta to this at each render/time step. You need to make sure that when α exceeds 360 degrees, you wrap around appropriately. You will have to fiddle a little to get the rotation to be as fast or slow as you like.

The Moon similarly rotates about the Earth, specified as some rotation angle, β, that changes at each time step, in the Eart-Moon coordinate system. This rotation is in the same plane as the rotation for the Sun-Earth.

The Moon should have a different rotation speed from the Sun-Earth rotation speed.

The net effect of implementing the required transformations is that at each draw call, when you correctly transform the models, the Earth will appear to rotate about the Sun with a certain speed, and simultaneously, the Moon will appear to rotate about the Earth at a different speed. Note that you will need to use scaling and translation in addition to rotation.

You should also add key presses to speed up/slow each rotation (independently), as

well as a key to start/stop the animation.

Note that, in OpenGL, you must redraw *everything* that needs to appear in the scene, for each frame generated, so make sure your draw calls are set up correctly. Usually these are triggered by the system at a desired frame rate.

Note that in this assignment, because we have one model, we only need to store the geometry for that single model. The shared sphere model is then correctly transformed - via matrix/matrices - prior to drawing the desired size, location etc. YOU SHOULD NOT MODIFY THE INPUT GEOMETRY (VERTICES). Once the VBO is set up, do not touch those values again.

3 Spherical planets

Note that your planets must appear as circular discs when projected, not ellipses! Set up your *camera projection type* accordingly.

4 Plagiarism

Due to the wide availability of OpenGL code around the internet very strict penalties will apply for any code that is found to not be your own. We will examine these assignments carefully so please submit code that you have written and acknowledge any external sources (you can only receive credit for code you have written, unless told otherwise).

5 Submission and Marking

This practical should be submitted as a single zip file which contains all code, accompanied by compilation/installation instructions for either Windows 10/11 or Ubuntu/Linux (Makefile for C++ OR venv install/execution scripts for Python).
MacOS is a little problematic since the TA only has access to Windows/Linux – it should be possible to generate a Makefile or Visual Studio (for Mac) project file that works, but you will need to ask him about that. If you use Xcode, that may be harder but it should be possible to coax it to export a project that VS for Mac can load. Using a Python implementation for Mac would like solve most issues, but if you prefer to use C++, then you'd need to explore the options above.

The compilation/build/run instructions should be contained in a README file, included as part of the submission. Failure to provide this setup information will lead to a 20% penalty.

Avoid submitting OBJ models unless they are not the ones provided (this bloats the submission size).

Evaluation will be primarily through a demonstration of your code in operation, with a significant portion of the marks coming from a code walkthrough with the TA. You will need to explain what more complex bits of OpenGL/shader code (not

scaffold code that we provide) are doing and why you have used them.

The demo evaluation will check to see that you have met the core requirements.

You will be required to download your code on your laptop to demonstrate that it works to the TA (although your code should be compilable for him – if you absolutely can't get that to work then this will ensure you still get most of the marks).

The TA will first try to compile you code off-line to do a preliminary assessment and put together any questions they may have for you. You will then need to set up a demo time (no more than 10 minutes) during which you show all the features (this should be quick) and during which they will ask you various questions about implementation.

Code Demo: Core Functionality [15 marks]

Feature	Max
Load model and display (at least 1 model)	0 – 1
Camera remains outside all models	0 – 1
Camera shows whole scene	0 – 1
Modelling:	
- Unique colour per model	0 – 1
- Models do not overlap	0 – 1
- Models sized as required (S >> E >> M)	0 – 1
- Models rotate about each other correctly	0 – 5
- Earth and Moon rotate at different speeds	0 – 2
Key Press Input:	
- Start/stop animation	0 – 1
- Change Rotation speed independently	0 – 1

Code Demo: Code Review/explanation [5 marks]

You need to explain what the various parts of your program are doing – *in detail.* The scaffold code can mostly be ignored, but we assume that all other code was written by you, so you should be able to explain its purpose clearly.

0: little understanding shown of OpenGL Code used in prac
[2,3,4]: award marks based on clarity of explanation (3 is average)
5: all code explained clearly; no misconceptions or misunderstandings

6 **Deadline:** Hand in assignment by 10.00AM 8 May, 2026

7 Useful Resources

1. https://learnopengl.com/
2. https://www.opengl.org/
3. https://libsdl.org/download-2.0.php
4. https://wiki.libsdl.org/
5. https://glm.g-truc.net/0.9.9/index.html
6. http://www.opengl-tutorial.org/
7. http://pyopengl.sourceforge.net/
8. https://www.pygame.org/docs/
9. https://pyrr.readthedocs.io/en/latest/